Experiment No.: 05
Title: Effect of Feature Scaling on Gradient Descent Convergence in Multiple Linear Regression

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 05**

Aim: To implement gradient descent for multiple linear regression and demonstrate the effect of feature scaling on its convergence behaviour.

Resources needed: Python 3.6 or later, NumPy, Matplotlib, Scikit-learn (for scaling)

Pre Requisite: Functions, Graphs, Statistics

Theory:

Gradient Descent is an optimization algorithm used to minimize the cost function in regression models by iteratively updating parameters in the direction of the steepest descent. In Multiple Linear Regression, the target variable is predicted as a linear combination of multiple input features. When features have vastly different ranges (e.g., hours studied [0–10] vs. marks in previous exam [0–100]), gradient descent can behave poorly, resulting in slow or erratic convergence.

Why Feature Scaling?

Feature scaling ensures all input features contribute equally to the gradient updates normalizing their ranges. This improves:

- Training speed (faster convergence)
- Stability (smoother descent path)
- Numerical accuracy

Common Scaling Techniques:

- Min-Max Scaling: Transforms features to a fixed range, usually [0, 1]
- Standardization: Centers data around mean 0 with unit variance

---

Procedure:

1. Generate a synthetic dataset with two features having different numerical ranges.
2. Implement multiple linear regression using gradient descent:
   - ○ Once without feature scaling
   - ○ Once with feature scaling
3. Track the cost function over iterations.
4. Plot and compare the convergence behaviors:
   - ○ Observe zig-zag behavior without scaling
   - ○ Observe smoother descent with scaling

Results: (Steps with screenshots)

Plots clearly demonstrate:

- Without Feature Scaling: Cost reduces slowly and irregularly, showing zig-zag behavior in parameter space.
- With Feature Scaling: Cost decreases smoothly, leading to faster and more stable convergence.

Code:

```
"""Compare gradient-descent convergence with and without feature scaling.

Requirements: numpy, matplotlib, scikit-learn
Run: python gradient_descent_feature_scaling.py
"""

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from pathlib import Path


def add_intercept(X):
    """Return X with a leading column of ones for the intercept."""
    return np.c_[np.ones(X.shape[0]), X]


def cost(X, y, theta):
    """Mean-squared-error cost J(theta) = 1/(2m) * sum(error^2)."""
    error = X @ theta - y
    return (error @ error) / (2 * len(y))


def gradient_descent(X, y, learning_rate, iterations):
    """Fit linear regression using batch gradient descent.

    Returns the fitted parameters, cost after every update, and the complete
    parameter history (useful for visualising the route through parameter space).
    """
    m, n = X.shape
    theta = np.zeros(n)
    cost_history = []
    theta_history = [theta.copy()]

    for _ in range(iterations):
        gradient = (X.T @ (X @ theta - y)) / m
        theta -= learning_rate * gradient
        cost_history.append(cost(X, y, theta))
        theta_history.append(theta.copy())

    return theta, np.asarray(cost_history), np.asarray(theta_history)


def scaled_to_original_theta(theta_scaled, scaler):
    """Convert parameters learned on standardized features back to raw units."""
    coefficients = theta_scaled[1:] / scaler.scale_
    intercept = theta_scaled[0] - np.sum(theta_scaled[1:] * scaler.mean_ /
scaler.scale_)
```

```
    return np.r_[intercept, coefficients]


def main():
    # Synthetic data: the feature ranges deliberately differ by a factor of 10.
    rng = np.random.default_rng(7)
    m = 250
    hours_studied = rng.uniform(0, 10, m)       # range approximately 0--10
    previous_exam_mark = rng.uniform(0, 100, m) # range approximately 0--100
    X_raw = np.c_[hours_studied, previous_exam_mark]
    y = 12 + 3.5 * hours_studied + 0.65 * previous_exam_mark + rng.normal(0, 3, m)

    # Same number of updates, but the raw features require a much smaller step.
    # A learning rate suitable for standardized data would make raw data diverge.
    iterations = 1_500
    X_unscaled = add_intercept(X_raw)
    theta_unscaled, cost_unscaled, path_unscaled = gradient_descent(
        X_unscaled, y, learning_rate=0.00015, iterations=iterations
    )

    scaler = StandardScaler()
    X_scaled_features = scaler.fit_transform(X_raw)
    X_scaled = add_intercept(X_scaled_features)
    theta_scaled, cost_scaled, path_scaled = gradient_descent(
        X_scaled, y, learning_rate=0.08, iterations=iterations
    )

    # Convert scaled-model history to raw coefficient units for a fair path plot.
    path_scaled_raw = np.array([
        scaled_to_original_theta(theta, scaler) for theta in path_scaled
    ])

    print("True model:       y = 12 + 3.5(hours) + 0.65(previous mark) + noise")
    print("Unscaled model:  intercept={:.3f}, hours={:.3f},
mark={:.3f}".format(*theta_unscaled))
    print("Scaled model:    intercept={:.3f}, hours={:.3f}, mark={:.3f}".format(
        *scaled_to_original_theta(theta_scaled, scaler)
    ))
    print("Final cost (unscaled): {:.4f}".format(cost_unscaled[-1]))
    print("Final cost (scaled):   {:.4f}".format(cost_scaled[-1]))

    # Cost comparison and a coefficient-space trace.
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    axes[0].plot(cost_unscaled, label="Without scaling (alpha = 0.00015)",
linewidth=2)
    axes[0].plot(cost_scaled, label="With standardization (alpha = 0.08)",
linewidth=2)
    axes[0].set_yscale("log")
    axes[0].set_xlabel("Iteration")
```

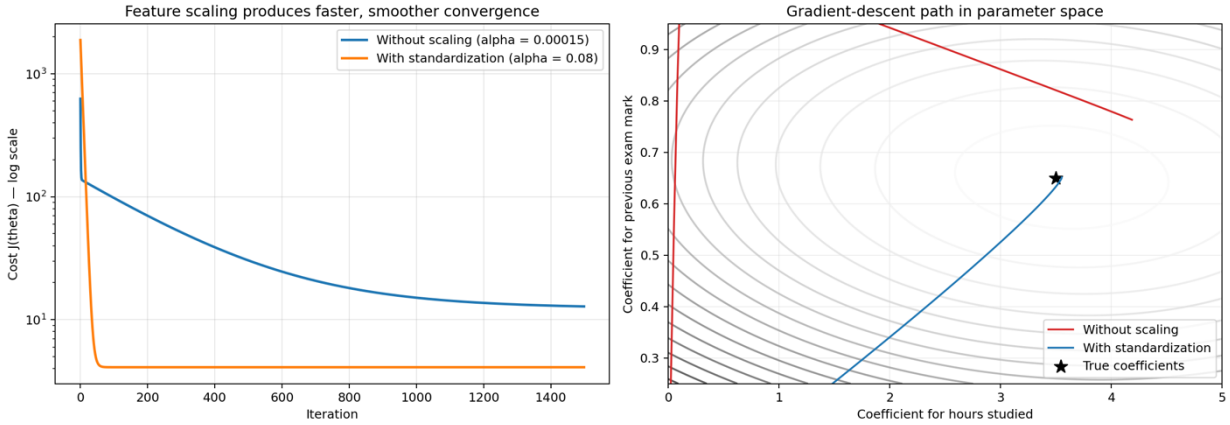
```
    axes[0].set_ylabel("Cost J(theta) — log scale")
    axes[0].set_title("Feature scaling produces faster, smoother convergence")
    axes[0].grid(alpha=0.3)
    axes[0].legend()

    # Plot both paths against cost contours in the two coefficient dimensions.
    # Intercept is optimised analytically for each pair of coefficients here.
    w_hours = np.linspace(0, 5, 120)
    w_mark = np.linspace(0.25, 0.95, 120)
    W_hours, W_mark = np.meshgrid(w_hours, w_mark)
    contour_cost = np.empty_like(W_hours)
    for i in range(W_hours.shape[0]):
        for j in range(W_hours.shape[1]):
            residual_without_intercept = y - (W_hours[i, j] * hours_studied +
                                              W_mark[i, j] * previous_exam_mark)
            best_intercept = residual_without_intercept.mean()
            contour_cost[i, j] = np.mean((best_intercept + W_hours[i, j] *
hours_studied +
                                          W_mark[i, j] * previous_exam_mark - y) **
2) / 2

    axes[1].contour(W_hours, W_mark, contour_cost, levels=18, cmap="Greys",
alpha=0.7)
    axes[1].plot(path_unscaled[:, 1], path_unscaled[:, 2], color="tab:red",
                 label="Without scaling", linewidth=1.5)
    axes[1].plot(path_scaled_raw[:, 1], path_scaled_raw[:, 2], color="tab:blue",
                 label="With standardization", linewidth=1.5)
    axes[1].scatter(3.5, 0.65, c="black", marker="*", s=120, label="True
coefficients")
    axes[1].set_xlim(0, 5)
    axes[1].set_ylim(0.25, 0.95)
    axes[1].set_xlabel("Coefficient for hours studied")
    axes[1].set_ylabel("Coefficient for previous exam mark")
    axes[1].set_title("Gradient-descent path in parameter space")
    axes[1].grid(alpha=0.2)
    axes[1].legend()

    fig.tight_layout()
    output_dir = Path("outputs")
    output_dir.mkdir(exist_ok=True)
    fig.savefig(output_dir / "feature_scaling_convergence.png", dpi=180,
bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```



Questions:

1. Why does gradient descent behave erratically without feature scaling?
2. What could happen if the learning rate is large and features are not scaled?
3. What are the benefits of vectorized gradient descent?

Answers

1. Without feature scaling, large-range features produce much larger gradient values than small-range features. Updates become uneven, causing the optimization path to oscillate or zig-zag across the cost surface. Gradient descent must use a very small learning rate, making convergence slow and unstable.

2. With unscaled features and a large learning rate, parameter updates can overshoot the minimum of the cost function. The cost may oscillate, increase repeatedly, or diverge to extremely large values. This prevents the model from learning accurate coefficients and can cause numerical instability.

3. Vectorized gradient descent uses matrix operations to calculate predictions, errors, gradients, and parameter updates simultaneously for all training examples. It removes explicit loops, makes code shorter and clearer, improves computational speed, and efficiently uses optimized numerical libraries such as NumPy.

Conclusion:

The experiment shows that feature scaling is essential for efficient gradient descent in multiple linear regression. When the features had different numerical ranges, gradient descent converged slowly and irregularly. After applying standardization, the cost function decreased smoothly and reached a lower value much faster. Thus, scaling improves the speed, stability, and accuracy of the regression model.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

Websites:

1. http://www.vmware.com/in
2. http://www.xenproject.org